

1. ¿Que es un ciclo de vida de desarrollo de software y cual es su propósito?
2. ¿Cuales son las principales etapas de un ciclo de vida de desarrollo de software?
3. ¿Porque es importante el SDLC?

R1) El ciclo de vida del desarrollo de software (SDLC) es un proceso rentable y eficiente en términos de tiempo empleado por los equipos de desarrollo para diseñar y crear software de alta calidad. El objetivo del SDLC es minimizar los riesgos del proyecto por medio de una planificación anticipada que permita que el software cumpla las expectativas del cliente durante la fase de producción y posteriormente. Esta metodología establece una serie de pasos que dividen el proceso de desarrollo de software en tareas que se pueden asignar, completar y medir

La metodología del ciclo de vida del desarrollo de software (SDLC) ofrece un marco de administración sistemático con entregas específicas en cada etapa del proceso de desarrollo de software.

R2)1. Planificación

La **fase de planificación del proyecto** establece los objetivos y el alcance de un proyecto de desarrollo de software.

El equipo de desarrollo de software comienza con una lluvia de ideas sobre los detalles generales del proyecto. El equipo podría centrarse en ideas como el problema o caso de uso que resolverá el software, quién lo utilizará y cómo podría interactuar con otras aplicaciones y sistemas.

Los desarrolladores también pueden solicitar la opinión de otras partes interesadas, como analistas de negocio, gerentes de línea de negocio y clientes internos y externos. Asimismo, pueden utilizar herramientas de investigación y codificación [de IA generativa](#) (Gen AI) para identificar requisitos o experimentar con nuevas funcionalidades del producto.

En general, la fase de planificación tiene como objetivo tanto definir los objetivos del proyecto como establecer lo que el proyecto *no* necesita, lo que ayuda a evitar que se vuelva demasiado extenso.

La fase de planificación suele generar un documento inicial de especificación de requisitos de software (SRS, por sus siglas en inglés). El SRS detalla las funciones del software, los recursos necesarios, los posibles riesgos y el cronograma del proyecto.

2. Análisis

Durante la **fase de análisis**, el equipo de desarrollo recopila y analiza información sobre los requisitos del proyecto. El análisis permite al equipo avanzar en el trabajo iniciado en la fase de planificación, pasando de una idea general a un plan de implementación práctico.

El análisis suele implicar la recopilación de requisitos de usuario, la realización de estudios de mercado y pruebas de viabilidad, la evaluación de prototipos y la asignación de recursos. Las partes interesadas pueden compartir datos sobre el rendimiento de la organización y de los clientes, información sobre desarrollos anteriores, requisitos de cumplimiento normativo y ciberseguridad de la empresa, y los recursos de TI disponibles.

Los desarrolladores de software podrían utilizar la IA generativa para procesar toda esta información. Por ejemplo, las herramientas de IA generativa podrían identificar tendencias en los comentarios de los usuarios o señalar posibles problemas de cumplimiento en las funciones propuestas.

Al finalizar la fase de análisis, los gestores de proyectos y los equipos de desarrollo comprenden plenamente el alcance del proyecto, sus especificaciones funcionales y técnicas, y cómo organizar las tareas y los flujos de trabajo del proyecto.

3. Diseño

La **fase de diseño** implica definir la arquitectura del proyecto. Los pasos principales incluyen la descripción de la navegación del software, las interfaces de usuario, el diseño de la base de datos y otros aspectos.

Los ingenieros de software revisan los requisitos para determinar la mejor manera de desarrollar el software deseado. Consideran cómo se integra el software en el entorno existente de aplicaciones y servicios de la organización, tanto ascendentes como descendentes, y cualquier otra dependencia que tenga.

El equipo de desarrollo también podría empezar a abordar las preocupaciones [de ciberseguridad](#) durante la fase de diseño mediante ejercicios de modelado de amenazas para identificar los riesgos potenciales del software. Por ejemplo, si se determina que el robo de identidad es un riesgo significativo, el equipo sabe que debe incorporar medidas [de autenticación](#) robustas en el diseño.

Muchas aplicaciones de software nuevas utilizan una arquitectura [de microservicios](#) , un enfoque [nativo de la nube](#) en el que una sola aplicación se compone de muchos componentes o servicios más pequeños, débilmente acoplados y desplegables de forma independiente.

Para organizar el flujo de desarrollo, los desarrolladores de software pueden utilizar un diseño modular. Los módulos de software son unidades de código independientes que

realizan una función específica. Estos módulos pueden conectarse para crear un software más grande. El diseño modular permite a los desarrolladores reutilizar módulos existentes y trabajar en varias partes del software simultáneamente, lo que reduce los cuellos de botella.

La fase de diseño suele culminar con la creación de un prototipo inicial o varios prototipos, que se pueden mostrar a las partes interesadas y a los usuarios finales para obtener comentarios. La IA genérica puede ayudar a acelerar la creación de prototipos. Por ejemplo, los desarrolladores podrían introducir diseños funcionales detallados y requisitos en una herramienta de IA para que esta genere un primer borrador del código del software.

El trabajo de la fase de diseño se recopila en un documento de diseño de software (SDD, por sus siglas en inglés), que se entrega a los desarrolladores como una hoja de ruta para que la utilicen durante la codificación.

4. Codificación

La **fase de codificación**, o **fase de desarrollo**, es cuando el equipo comienza a escribir el código y a construir el software, basándose en el SDD, el SRS y otras directrices creadas durante las fases anteriores.

Estos documentos ayudan a orientar a los desarrolladores de software a la hora de elegir el lenguaje de programación correcto, como [Java](#)™ o C++, y ayudan a los gestores de proyectos a dividir el proyecto en tareas de codificación más pequeñas e independientes.

Esta fase también implica la creación de cualquier sistema o interfaz adicional necesaria para que el software funcione correctamente, como páginas web o [interfaces de programación de aplicaciones \(API\)](#).

Dependiendo del modelo SDLC que sigan, algunos equipos podrían realizar revisiones de código y otras pruebas durante la fase de desarrollo. Estas pruebas podrían ayudar a identificar errores y otras vulnerabilidades en una etapa temprana del ciclo de vida del desarrollo de software.

Algunos desarrolladores utilizan ahora IA generativa para escribir código, lo que puede reducir el tiempo de desarrollo. Por ejemplo, en la [«programación intuitiva»](#), los desarrolladores describen en texto plano lo que quieren que haga el software, y la herramienta de IA crea los fragmentos de código adecuados. Los desarrolladores suelen tomar este código como punto de partida y lo perfeccionan posteriormente.

5. Pruebas

La **fase de pruebas** comienza una vez que el equipo de desarrollo ha creado un software funcional. Durante esta fase, el equipo busca oportunidades para eliminar errores y mejorar el producto final.

Un equipo de control de calidad puede realizar pruebas unitarias, de integración, de sistema, de aceptación y otros [tipos de pruebas](#) para garantizar que todas las partes del software funcionen según lo previsto. Estas pruebas ayudan a asegurar que el software cumpla con los requisitos del usuario y del negocio, y que funcione dentro del entorno de TI general de la organización.

Los evaluadores también examinan minuciosamente el software en busca de vulnerabilidades de seguridad, identifican cuándo y cómo se producen y documentan los hallazgos.

Los desarrolladores utilizan los resultados de las pruebas para corregir errores e implementar mejoras, y luego envían el software de vuelta para que se vuelva a probar.

Los equipos de desarrollo de software suelen utilizar métodos de prueba tanto manuales como [automatizados](#) . Las herramientas de IA pueden optimizar gran parte del proceso de pruebas; por ejemplo, generando casos de prueba y analizando patrones en los fallos para descubrir las causas raíz.

Muchos modelos SDLC utilizan pruebas continuas. Este enfoque implica que las pruebas no se limitan a una sola fase del SDLC, sino que el código se prueba a lo largo de todo el proceso de desarrollo de software.

6. Despliegue

En la **fase de despliegue** , el software, finamente ajustado, se implementa en el entorno de producción donde los usuarios pueden acceder a él.

El objetivo de esta fase no es simplemente poner el software en manos de los usuarios. Los desarrolladores quieren asegurarse de que los usuarios *comprendan* cómo usar el nuevo software y que se pueda implementar con una mínima interrupción de la [experiencia del usuario](#) o [de los flujos de trabajo](#) .

Los desarrolladores pueden implementar el software por fases, como una versión beta, donde un grupo limitado de usuarios prueba una versión preliminar, antes de su lanzamiento al público. Este enfoque permite al equipo observar el funcionamiento del software en un

entorno real antes de su disponibilidad general. Los equipos de desarrollo también pueden crear manuales, impartir sesiones de capacitación u ofrecer soporte técnico presencial a los usuarios.

7. Mantenimiento

El ciclo de vida del desarrollo de software (SDLC) no termina con la implementación del software. La **fase de mantenimiento** comprende el trabajo posterior a la implementación que realizan los equipos de software para garantizar su funcionamiento continuo: implementar actualizaciones y optimizaciones, realizar cambios imprevistos, probar parches, abordar nuevos casos de uso y corregir cualquier error que encuentren los usuarios.

El mantenimiento y el soporte continuos son esenciales para garantizar la durabilidad de cualquier software. Piénselo como el mantenimiento de una casa: con el tiempo, algunas piezas se desgastarán o se estropearán. Estas, a su vez, se reemplazarán y, con suerte, se mejorarán.

En algunos modelos de desarrollo, como los modelos [DevOps](#) , los equipos de desarrollo (Dev) y operaciones de TI (Ops) utilizan [la integración continua y la entrega continua \(CI/CD\)](#) . El código se agrega continuamente a la base de código a medida que se escribe, se prueba continuamente y se implementa automáticamente en el entorno de producción. En DevOps, el mantenimiento es una actividad continua, no una fase diferenciada.

R3)

El desarrollo de software puede ser difícil de administrar debido a los requisitos cambiantes, los avances de la tecnología y la colaboración interfuncional. La metodología del ciclo de vida del desarrollo de software (SDLC) ofrece un marco de administración sistemático con entregas específicas en cada etapa del proceso de desarrollo de software. Como resultado, todas las partes interesadas establecen por adelantado los objetivos y requisitos de desarrollo del software y también cuentan con una planificación para conseguirlo.

A continuación, se indican algunas ventajas del SDLC:

- Mayor visibilidad del proceso de desarrollo para todas las partes interesadas implicadas
- Una estimación, planificación y programación eficientes
- Mejoras en la administración de riesgos y estimación de costos
- Entregas de software sistemáticas y mayor satisfacción de los clientes